

SMART HOME AUTOMATION CODE

C++

```
/*  
 * Project: Smart Home Automation System  
 * File Name: smart_home_automation.cpp  
 * Platform: ESP32 Edge Gateway  
 * Sensors: DHT11 (Temp/Humidity), LDR (Light), PIR (Motion)  
 * Actuators: 4-Channel Relay Module  
 * Description: Embedded C++ firmware for real-time telemetry and automation control.  
 */  
  
#include <WiFi.h>  
#include <PubSubClient.h>  
#include <DHT.h>  
  
// ===== USER CONFIGURATION =====  
  
const char* ssid = "YOUR_WIFI_SSID";  
const char* password = "YOUR_WIFI_PASSWORD";  
const char* mqtt_server = "broker.hivemq.com";  
const int mqtt_port = 1883;  
  
// ===== HARDWARE PIN MAPPING =====  
  
#define DHTPIN 4  
#define DHTTYPE DHT11  
#define LDRPIN 34  
#define PIRPIN 35  
  
#define RELAY1_PIN 25 // Light Control  
#define RELAY2_PIN 26 // Fan Control
```

```

#define RELAY3_PIN 27 // Appliance 3
#define RELAY4_PIN 14 // Appliance 4

// ===== INSTANCES & VARIABLES =====

DHT dht(DHTPIN, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastMsg = 0;
const long interval = 5000; // Publish telemetry every 5 seconds

// ===== SETUP FUNCTIONS =====

void setup_wifi() {
    delay(10);
    Serial.println();
    Serial.print("Connecting to WiFi: ");
    Serial.println(ssid);

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi Connected!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length) {
    String message;
    for (unsigned int i = 0; i < length; i++) {

```

```
    message += (char)payload[i];  
}
```

```
Serial.print("Message arrived [");  
Serial.print(topic);  
Serial.print("]: ");  
Serial.println(message);
```

```
// Relay 1 Control
```

```
if (String(topic) == "home/relay1") {  
    digitalWrite(RELAY1_PIN, message == "ON" ? LOW : HIGH); // Active LOW Relay  
}
```

```
// Relay 2 Control
```

```
if (String(topic) == "home/relay2") {  
    digitalWrite(RELAY2_PIN, message == "ON" ? LOW : HIGH);  
}  
}
```

```
void reconnect() {
```

```
    while (!client.connected()) {  
        Serial.print("Attempting MQTT connection...");  
        String clientId = "ESP32Client-" + String(random(0xffff), HEX);
```

```
        if (client.connect(clientId.c_str())) {  
            Serial.println("Connected to MQTT Broker!");  
            client.subscribe("home/relay1");  
            client.subscribe("home/relay2");
```

```
        } else {  
            Serial.print("Failed, rc=");  
            Serial.print(client.state());  
            Serial.println(" Retrying in 5 seconds...");  
            delay(5000);
```

```

    }
}

// ===== MAIN SETUP & LOOP =====

void setup() {
    Serial.begin(115200);

    // Initialize Sensors
    dht.begin();
    pinMode(LDRPIN, INPUT);
    pinMode(PIRPIN, INPUT);

    // Initialize Relays (Active LOW setup)
    pinMode(RELAY1_PIN, OUTPUT);
    pinMode(RELAY2_PIN, OUTPUT);
    pinMode(RELAY3_PIN, OUTPUT);
    pinMode(RELAY4_PIN, OUTPUT);

    digitalWrite(RELAY1_PIN, HIGH);
    digitalWrite(RELAY2_PIN, HIGH);
    digitalWrite(RELAY3_PIN, HIGH);
    digitalWrite(RELAY4_PIN, HIGH);

    setup_wifi();
    client.setServer(mqtt_server, mqtt_port);
    client.setCallback(callback);
}

void loop() {
    if (!client.connected()) {
        reconnect();
    }
}

```

```

}

client.loop();

unsigned long now = millis();
if (now - lastMsg > interval) {
    lastMsg = now;

    // Read Sensor Values
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();
    int ldrValue = analogRead(LDRPIN);
    int pirState = digitalRead(PIRPIN);

    if (isnan(temp) || isnan(hum)) {
        Serial.println("Failed to read from DHT sensor!");
        return;
    }

    // Print to Serial Monitor
    Serial.printf("Temp: %.1f°C | Humidity: %.1f%% | LDR: %d | Motion: %s\n",
        temp, hum, ldrValue, pirState ? "YES" : "NO");

    // Publish Telemetry Data to MQTT
    client.publish("home/temperature", String(temp).c_str());
    client.publish("home/humidity", String(hum).c_str());
    client.publish("home/light", String(ldrValue).c_str());
    client.publish("home/motion", pirState ? "DETECTED" : "CLEAR");

    // Automated Edge Logic
    if (pirState == HIGH && ldrValue < 1000) {
        digitalWrite(RELAY1_PIN, LOW); // Turn ON Light
        client.publish("home/relay1/status", "ON");
    }
}

```

}

}

}